

Assignment 14.4 Ai Assisted Coding

Htno:2303a51288

Btno:05

Task 1: Task 1: Responsive Homepage using HTML and CSS

Prompt: Generate a responsive homepage using HTML and CSS with a header, navigation menu, hero section, services section, and footer.

Use CSS media queries to make the layout responsive.

Codes: index ,style css

File Edit Selection View ... < > lab14.4_project

EXPLORER

LAB14.4_PROJECT

.vscode

index.html

style.css

OUTLINE

TIMELINE

index.html X # style.css

index.html > html > body > header

1 <!DOCTYPE html>

2 <html>

3 <head>

4 <title>My Startup</title>

5 <link rel="stylesheet" href="style.css">

6 </head>

7

8 <body>

9

10 <header>

11 <div class="logo">My Startup</div>

12

13 <nav>

14 Home

15 Services

16 About

17 Contact

18 </nav>

19 </header>

20

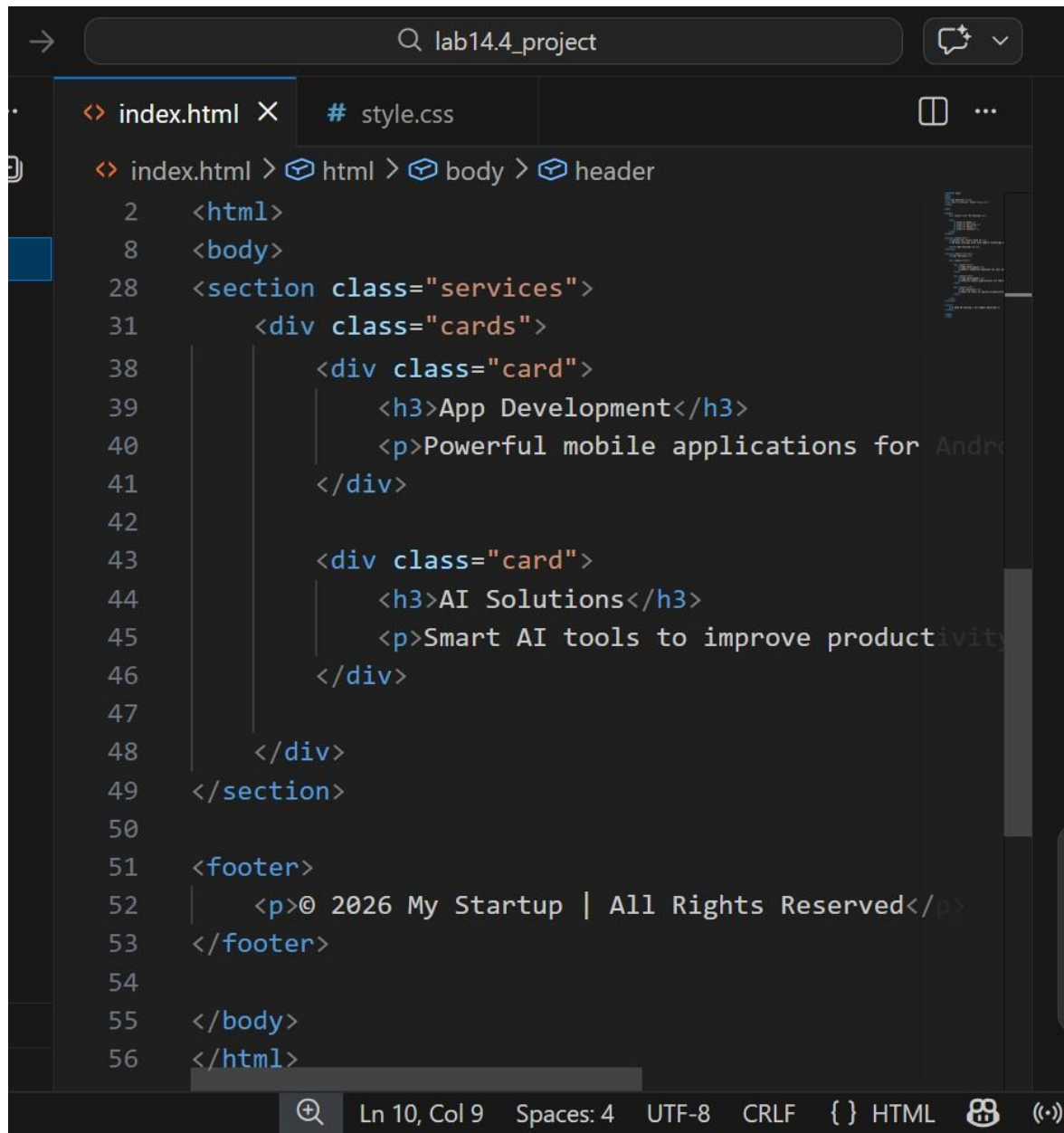
21 <section class="hero">

22 <h1>Build Your Future With Us</h1>

23 <p>We help startups grow with modern technology

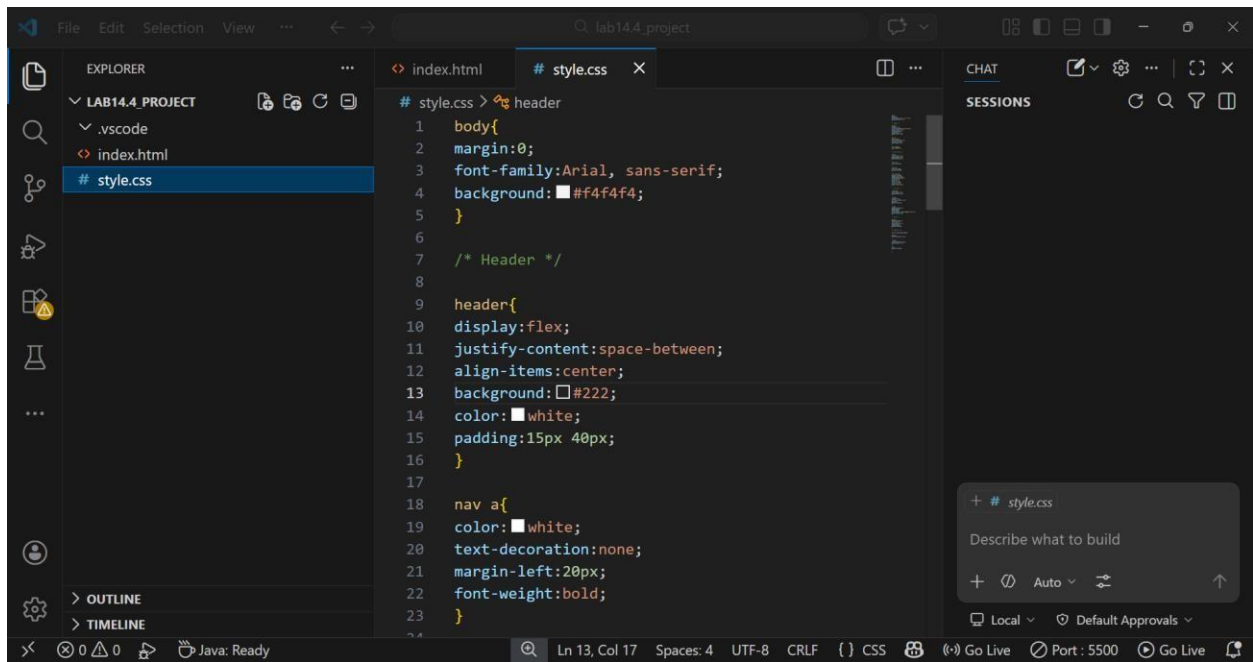
24

0 0 Java: Ready Ln 10, Col 9 Spaces: 4 UTF-8 CRLF {} HTML



```
→ lab14.4_project
index.html X # style.css
index.html > html > body > header
2  <html>
8  <body>
28 <section class="services">
31   <div class="cards">
38     <div class="card">
39       <h3>App Development</h3>
40       <p>Powerful mobile applications for Android</p>
41     </div>
42
43     <div class="card">
44       <h3>AI Solutions</h3>
45       <p>Smart AI tools to improve productivity</p>
46     </div>
47   </div>
48 </section>
49
50
51 <footer>
52   <p>© 2026 My Startup | All Rights Reserved</p>
53 </footer>
54
55 </body>
56 </html>
Ln 10, Col 9 Spaces: 4 UTF-8 CRLF { } HTML
```

Style css:

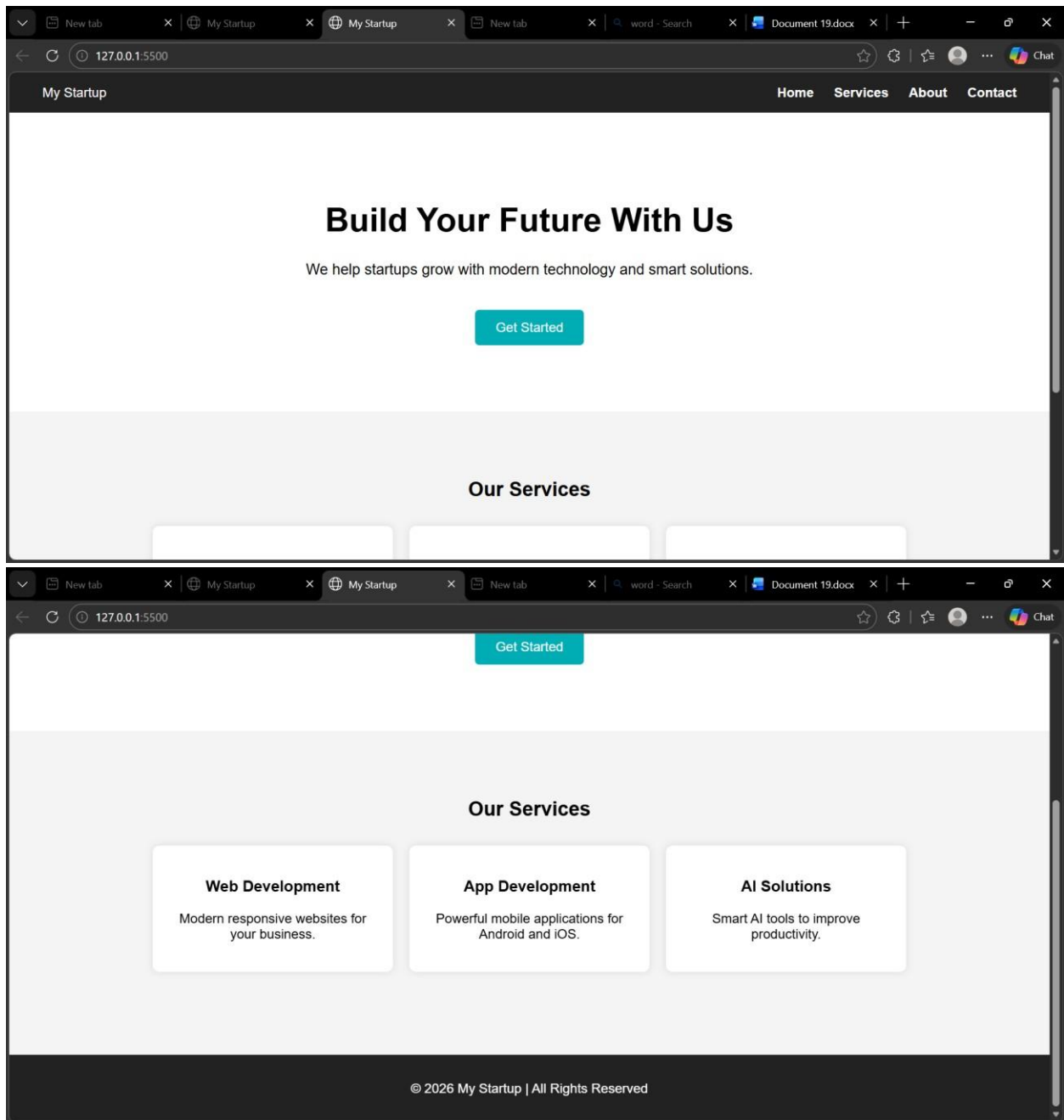


The image shows a code editor window with a dark theme. The top bar includes a search icon and the text 'lab14.4_project'. Below the top bar, there are two tabs: 'index.html' and '# style.css'. The '# style.css' tab is active, and the editor shows the following CSS code:

```
# style.css > header
70
71  .card{
72    background: white;
73    padding:20px;
74    width:250px;
75    box-shadow:0 0 10px rgba(0,0,0,0.1);
76    border-radius:8px;
77  }
78
79  /* Footer */
80
81  footer{
82    background: #222;
83    color: white;
84    text-align:center;
85    padding:15px;
86    margin-top:40px;
87  }
88
89  /* Responsive */
90
91  @media (max-width:768px){
92
93  header{
```

The status bar at the bottom shows 'Ln 13, Col 17', 'Spaces: 4', 'UTF-8', 'CRLF', and '{ } CS'.

Output:



Explanation:

-->This task creates a responsive homepage using HTML and CSS.

-->The webpage contains a header with navigation links, a hero section, --
>service cards, and a footer. CSS is used for styling and media queries are
used to make the webpage responsive for different screen sizes.

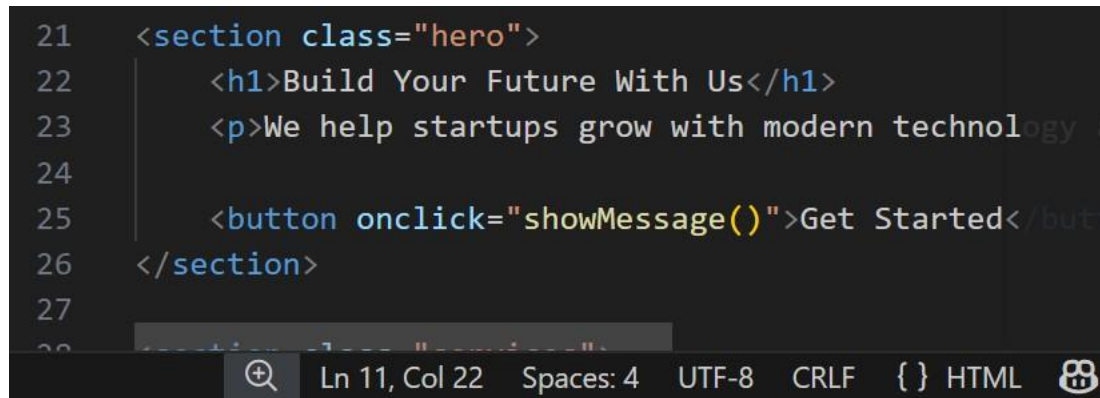
Task 2: Button Interaction using JavaScript

Prompt: Add a call-to-action button to the webpage.

When the user clicks the button, display an alert message using JavaScript without reloading the page.

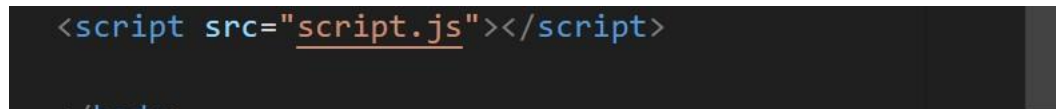
Codes: index.html – edited part

```
21 <section class="hero">
22   <h1>Build Your Future With Us</h1>
23   <p>We help startups grow with modern technology </p>
24
25   <button onclick="showMessage()">Get Started</button>
26 </section>
27
28 <section class="hero">
```

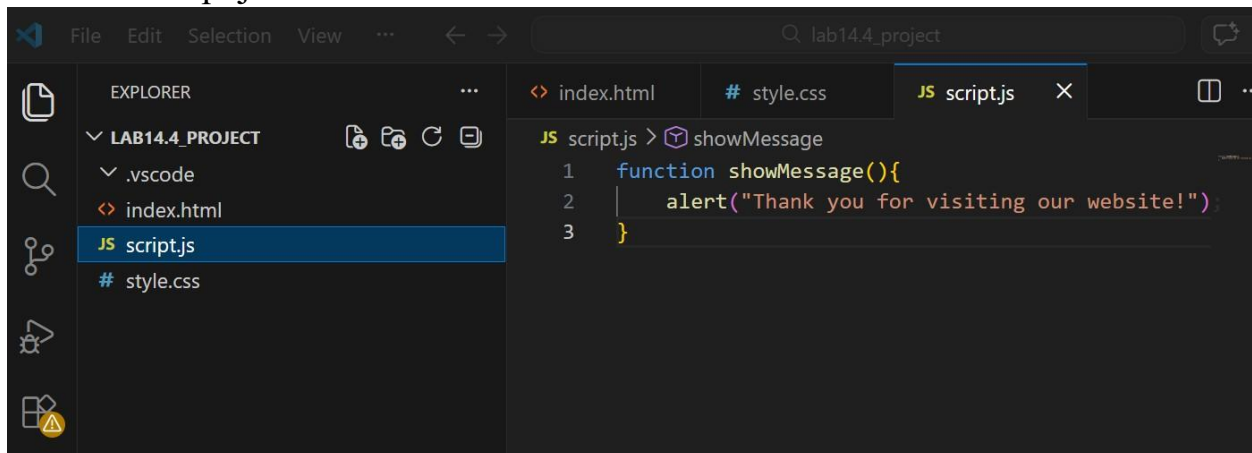


Java Script connection:

```
<script src="script.js"></script>
```

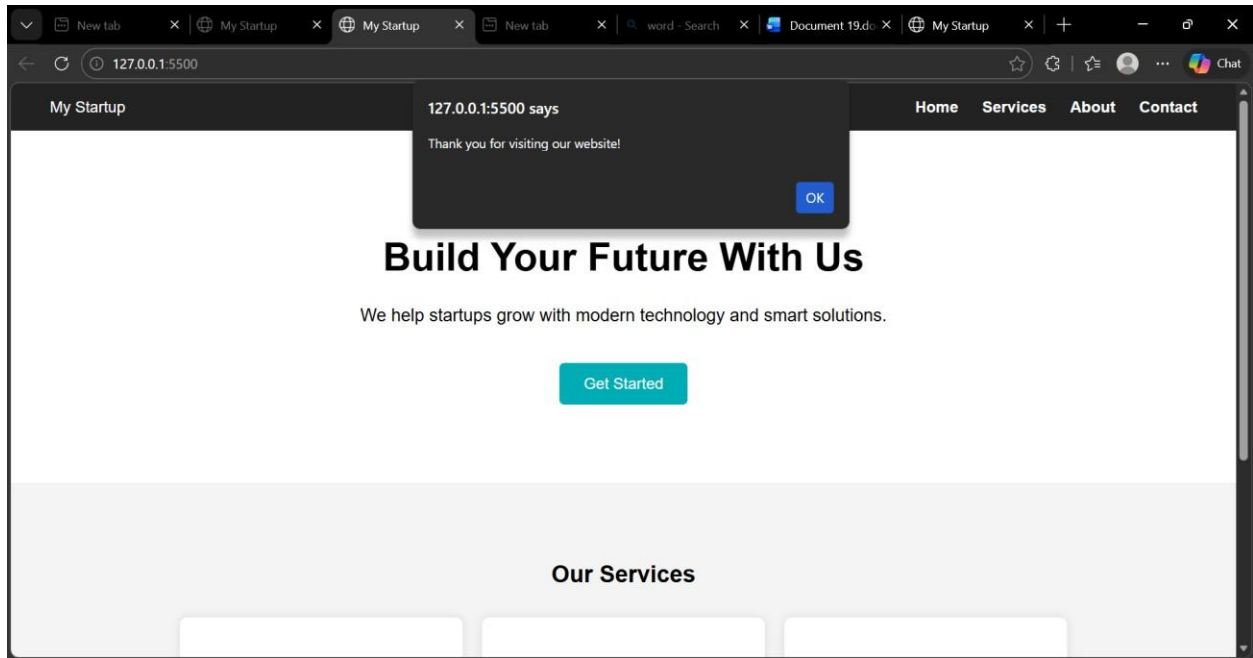


Add file :script.js



```
JS script.js > showMessage
1 function showMessage(){
2   alert("Thank you for visiting our website!");
3 }
```

Output:



Explanation:

The button interaction is implemented using JavaScript.

When the user clicks the Get Started button, the showMessage() function is triggered and an alert message is displayed on the screen.

Task 3: Contact Form Validation using JavaScript

Prompt: Create a contact form using HTML with fields for Name, Email, and Message.

Use JavaScript to validate the form by checking empty fields and verifying the email format before submission.

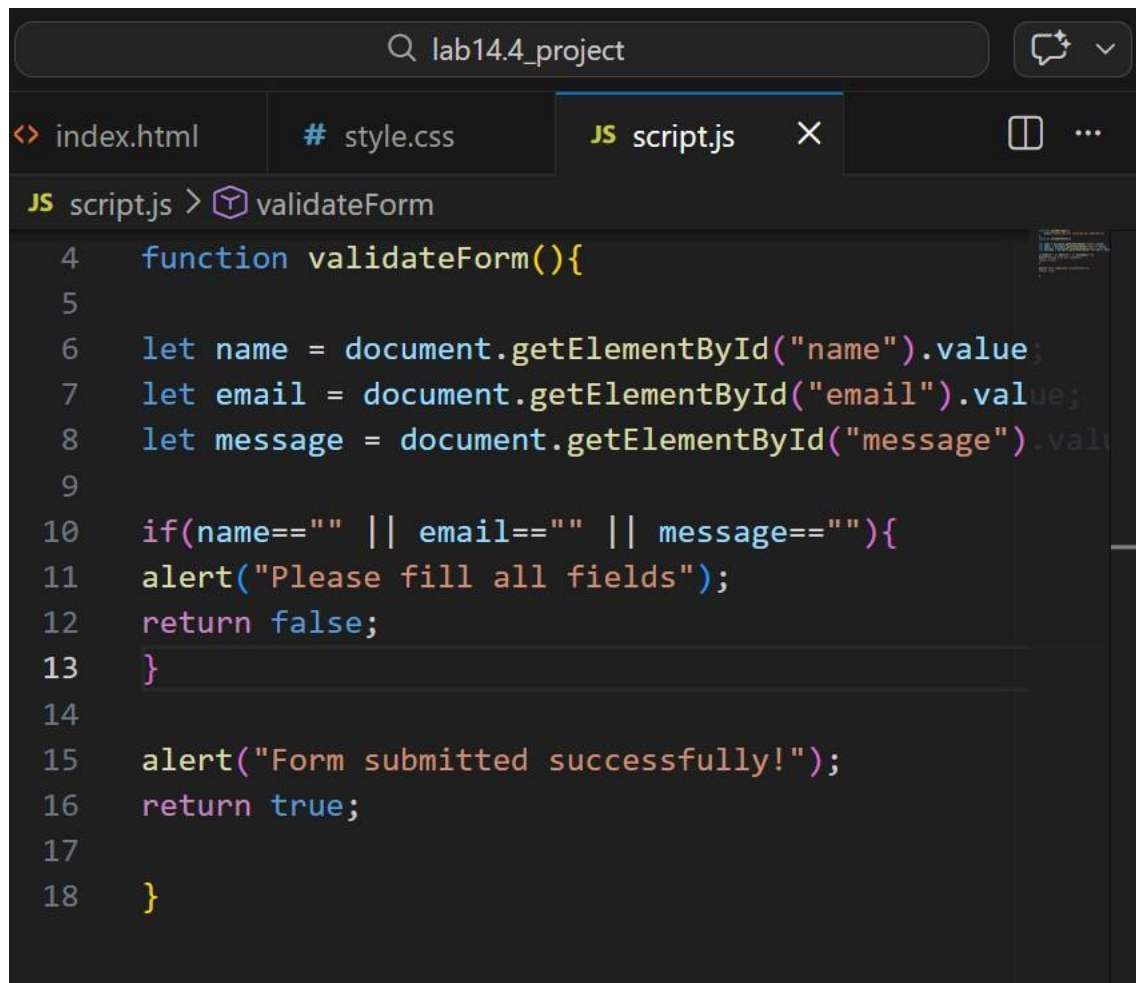
Codes: index.html – Contact Form Section

```
lab14.4_project

index.html X # style.css JS script.js

index.html > html > body
2  <html>
8  <body>
50 <section class="contact">
51
52 <h2>Contact Us</h2>
53
54 <form onsubmit="return validateForm()">
55
56 <input type="text" id="name" placeholder="Enter yo
57
58 <input type="email" id="email" placeholder="Enter
59
60 <textarea id="message" placeholder="Enter your mes
61
62 <button type="submit">Submit</button>
63
64 </form>
65
66 </section>
67 <footer>
68 |   <p>© 2026 My Startup | All Rights Reserved</p>
69 </footer>
70 <script src="script.js"></script>
71
```

2)script.js – add below previous code:

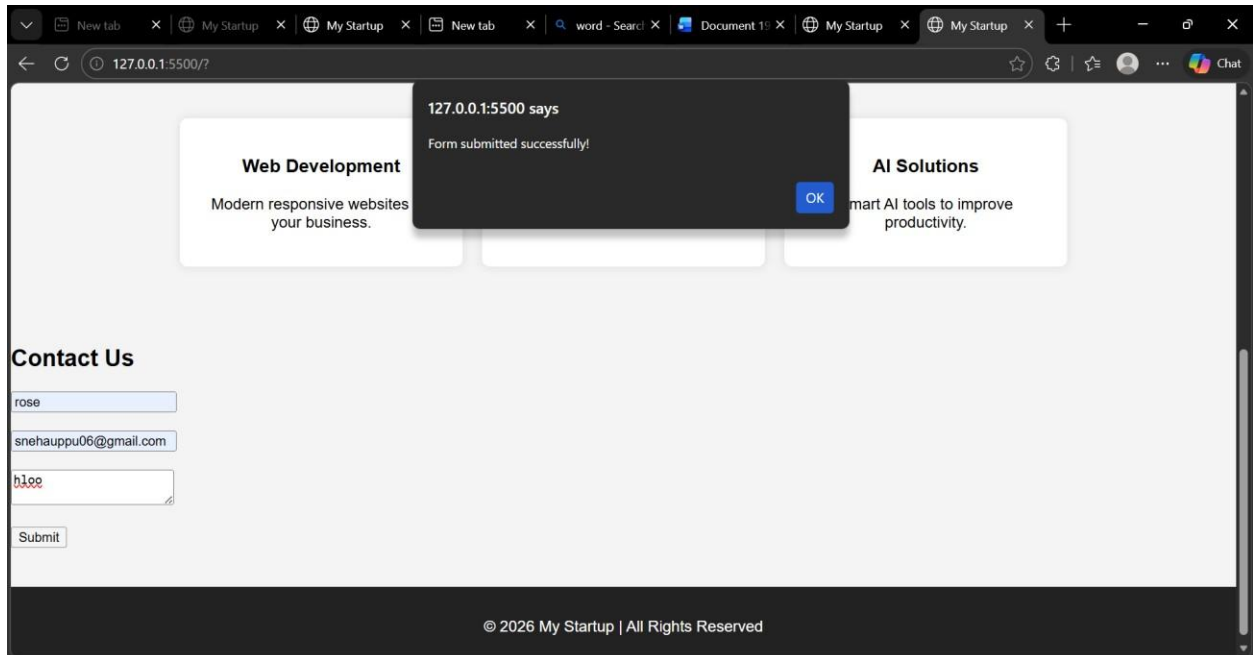


```
lab14.4_project

index.html # style.css JS script.js X
JS script.js > validateForm

4 function validateForm(){
5
6 let name = document.getElementById("name").value;
7 let email = document.getElementById("email").value;
8 let message = document.getElementById("message").value;
9
10 if(name=="" || email=="" || message==""){
11 alert("Please fill all fields");
12 return false;
13 }
14
15 alert("Form submitted successfully!");
16 return true;
17
18 }
```

Output:



Expalantion:

This task creates a **contact form with Name, Email, and Message fields**.

JavaScript validation checks whether the fields are empty and displays an alert message before submitting the form.

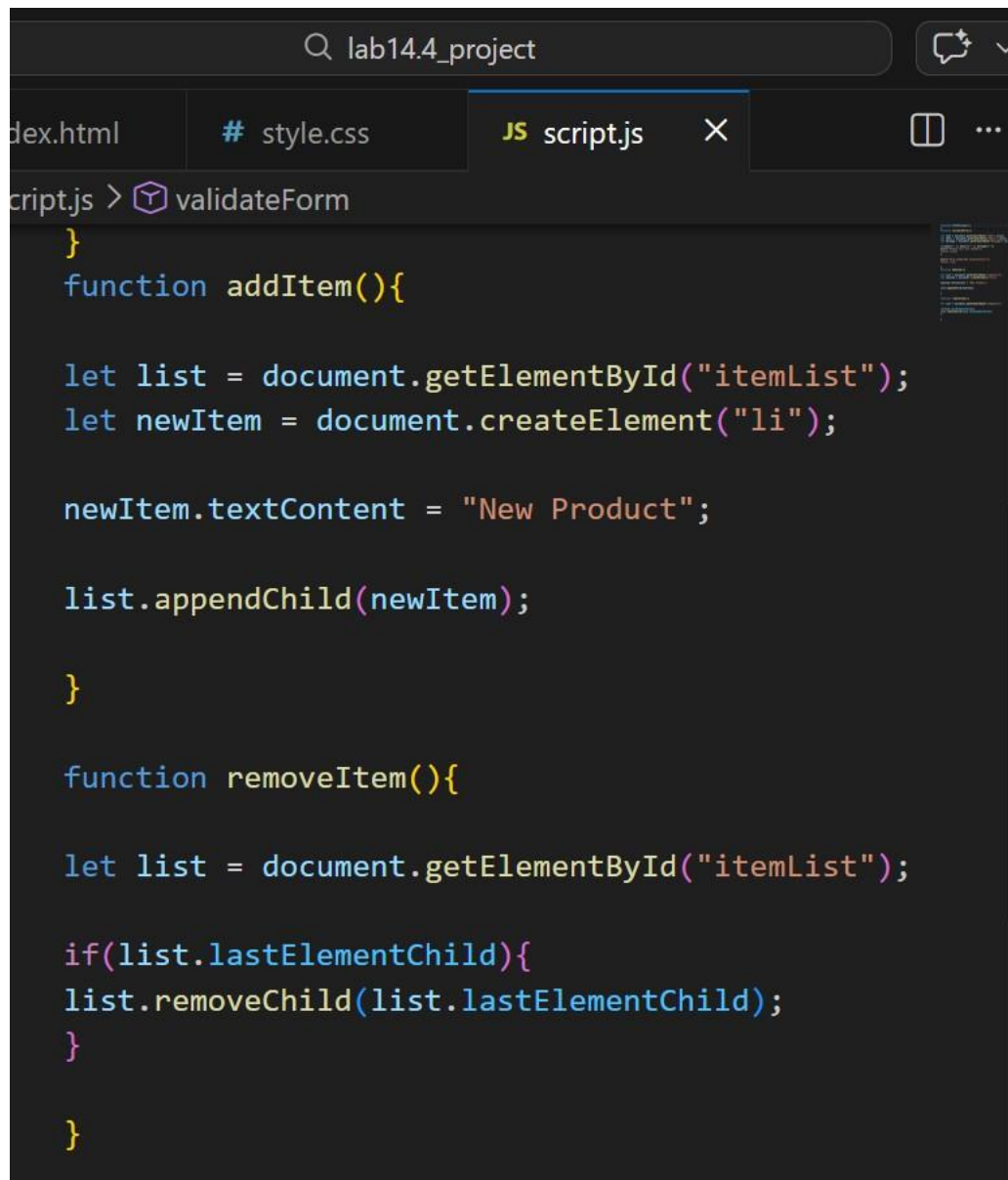
Task 4: Dynamic Product List using JavaScript

Prompt: Create a dynamic product list where users can **add new items and remove items** from the list using JavaScript buttons.

Code:

```
... index.html X # style.css JS script.js
index.html > html > body
2 <html>
8 <body>
67 <section class="list">
68
69 <h2>Product List</h2>
70
71 <ul id="itemList">
72 <li>Product 1</li>
73 <li>Product 2</li>
74 </ul>
75
76 <button onclick="addItem()">Add Item</button>
77 <button onclick="removeItem()">Remove Item</button>
78
79 </section>
80 <footer>
81 | <p>© 2026 My Startup | All Rights Reserved</p>
82 </footer>
83 <script src="script.js"></script>
84
85 </body>
86 </html>
```

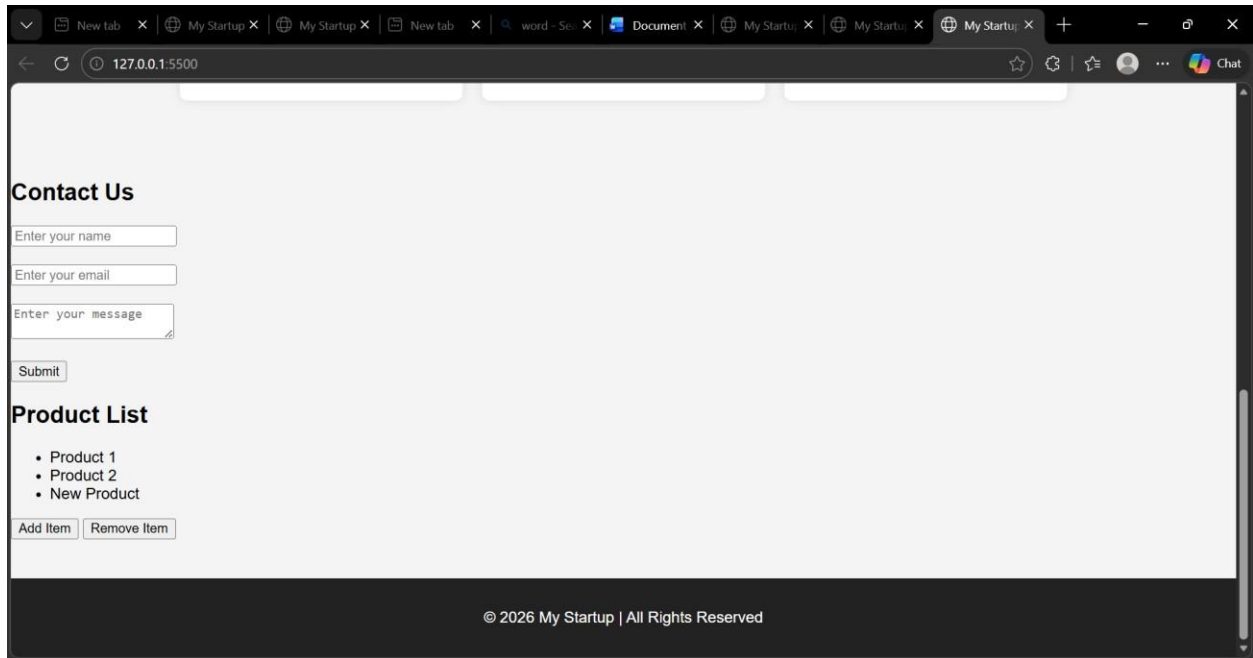
2)Java Script Add Item:



The image shows a web browser window with the address bar displaying "lab14.4_project". The browser has three tabs open: "dex.html", "# style.css", and "JS script.js". The "JS script.js" tab is active, and the browser's developer console is open, showing the "validateForm" function. The code in the console is as follows:

```
}  
function addItem(){  
  
    let list = document.getElementById("itemList");  
    let newItem = document.createElement("li");  
  
    newItem.textContent = "New Product";  
  
    list.appendChild(newItem);  
  
}  
  
function removeItem(){  
  
    let list = document.getElementById("itemList");  
  
    if(list.lastElementChild){  
        list.removeChild(list.lastElementChild);  
    }  
  
}
```

Output:



Explanation:

This program creates a dynamic product list where users can add or remove items using buttons. JavaScript DOM functions like `createElement()` and `appendChild()` are used to modify the list.

Task 5: Fetch Data using JavaScript API

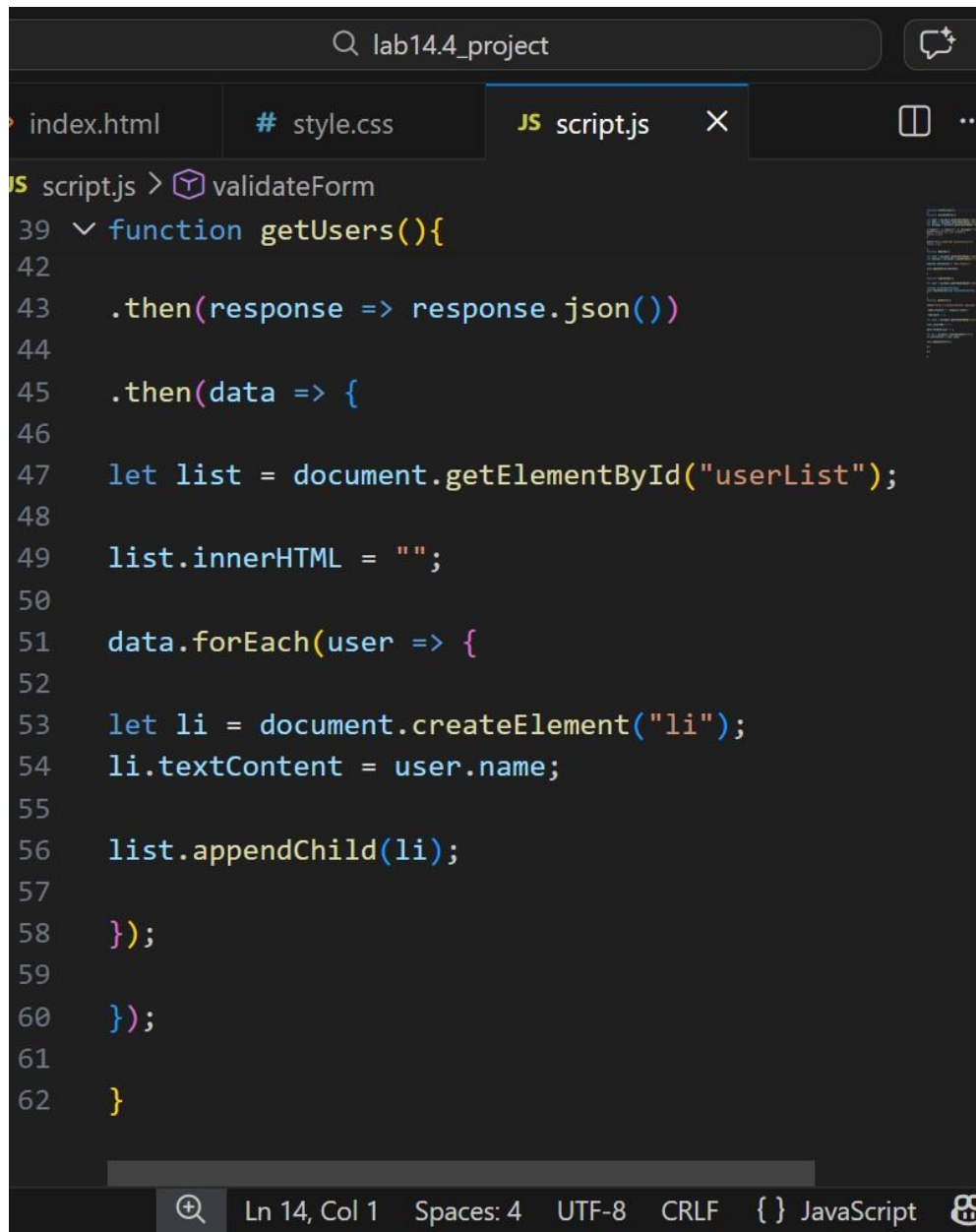
Prompt: Create a webpage that **fetches data from an API using JavaScript Fetch API and displays it on the webpage dynamically.**

Codes: Index .Html

```
78
79 </section>
80 <section class="api">
81
82 <h2>User Data</h2>
83
84 <button onclick="getUsers()">Load Users</button>
85
86 <ul id="userList"></ul>
87
88 </section>
89 <footer>
90 |   <p>© 2026 My Startup | All Rights Reserved</p>
91 </footer>
92 <script src="script.js"></script>
93
```

Ln 8, Col 7 Spaces: 4 UTF-8 CRLF {} HTML

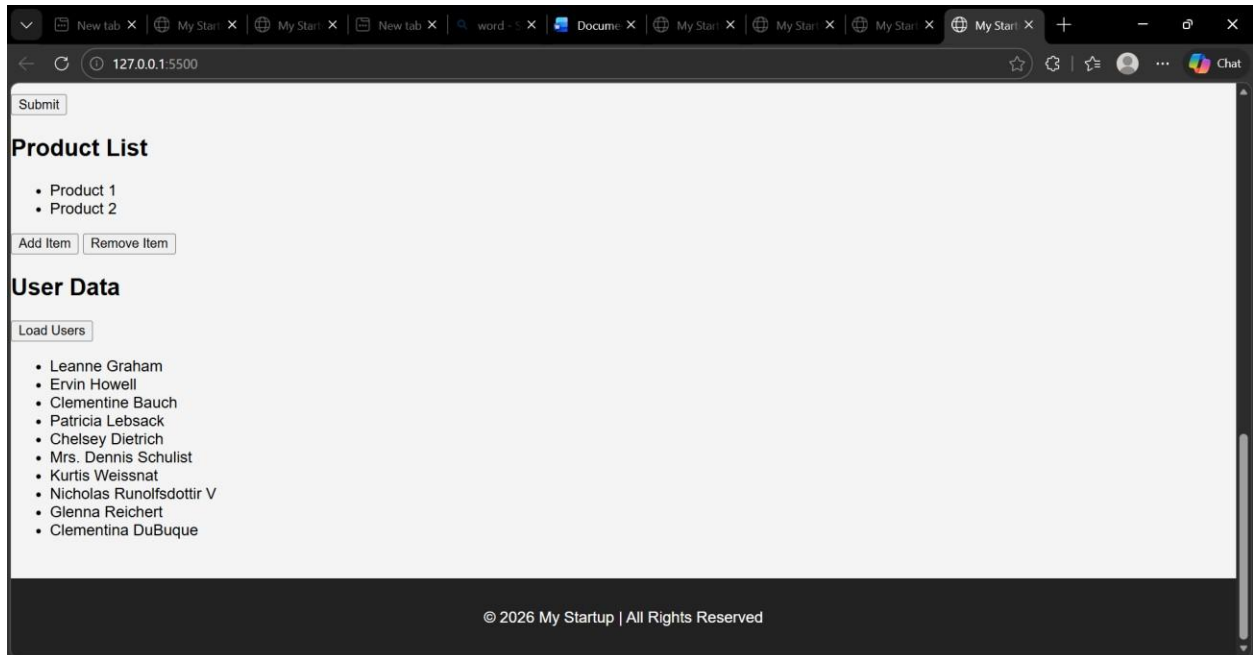
2)Java Script(Load Users):



```
lab14.4_project
index.html # style.css JS script.js
script.js > validateForm
39 function getUsers(){
42
43   .then(response => response.json())
44
45   .then(data => {
46
47     let list = document.getElementById("userList");
48
49     list.innerHTML = "";
50
51     data.forEach(user => {
52
53       let li = document.createElement("li");
54       li.textContent = user.name;
55
56       list.appendChild(li);
57
58     });
59
60   });
61
62 }
```

Ln 14, Col 1 Spaces: 4 UTF-8 CRLF { } JavaScript

Output:



Explanation:

The Fetch API is used to retrieve user data from an external API. JavaScript dynamically creates list elements and displays the user names on the webpage. The JavaScript code processes this data and dynamically creates list elements to display the user names on the webpage. This allows the webpage to update content without reloading the page.